

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA FACULTAD
DE INGENIERÍA EN SISTEMAS COMPUTACIONALES CENTRO
UNIVERSITARIO DE PETÉN UMG



Curso: Programación 1

Docente: Ing. Luis Antonio García Aguirre

Proyecto Final: Sistema de Gestión con C++ y MySQL para Servidor de videojuegos.

31 de mayo del 2026, Guatemala, Flores, Petén

Marcos David Barrios Roque - 1690-25-22591

Introducción:

Este documento presenta el Proyecto Final para el curso de Programación 1. El programa es un sistema de gestión desarrollado en C++ y conectado a una base de datos MySQL, diseñado para administrar la información de un servidor de minecraft.

Para lograr la conexión y manejar los datos, utilicé la librería EloquentORM del github. Con esto, el sistema permite realizar las cuatro operaciones básicas (CRUD): registrar, leer, actualizar y eliminar información directamente desde un menú en la consola. El proyecto utiliza dos tablas relacionadas (clanes y jugadores) para organizar a qué clan pertenece cada usuario, poniendo en práctica los conceptos de programación y bases de datos aprendidos en el semestre.

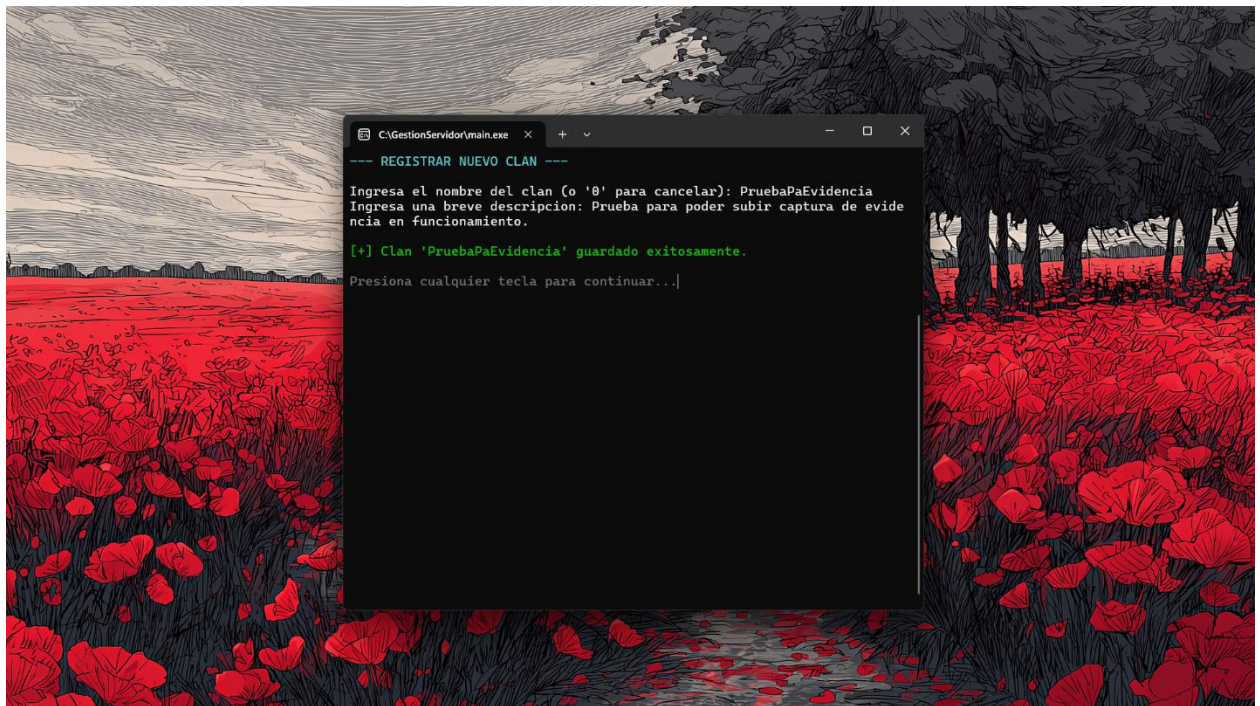
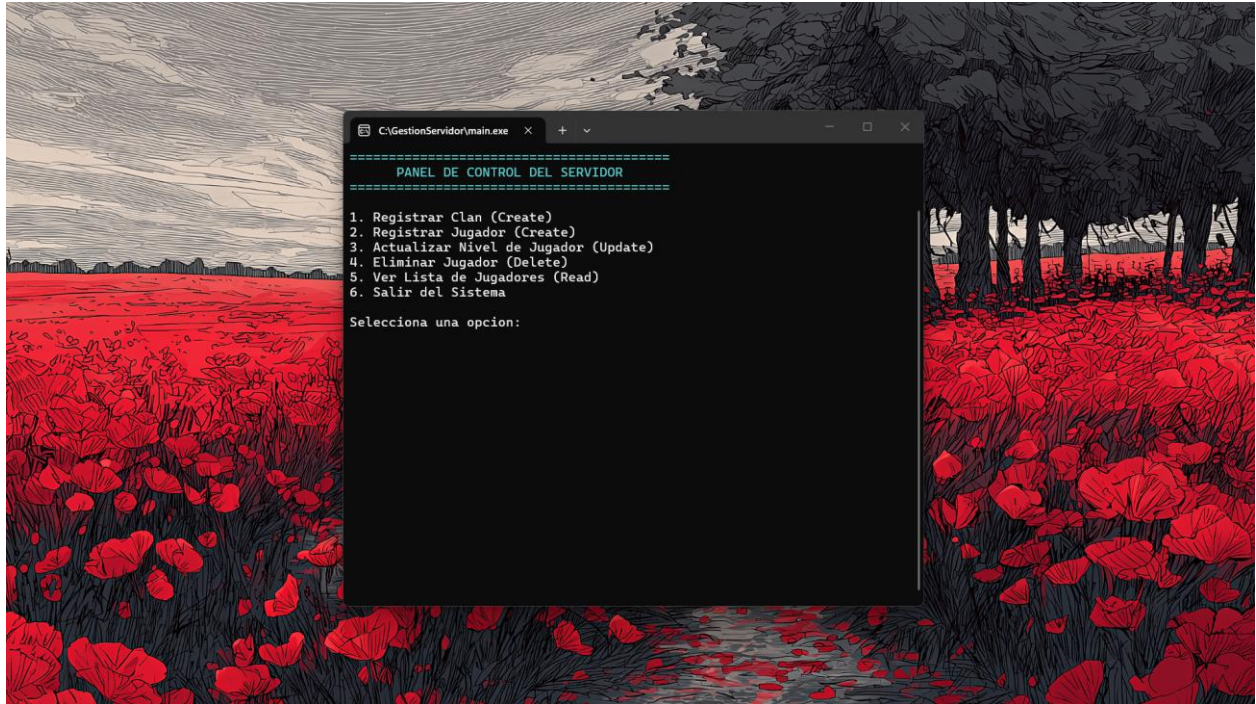
Estructura del Proyecto:

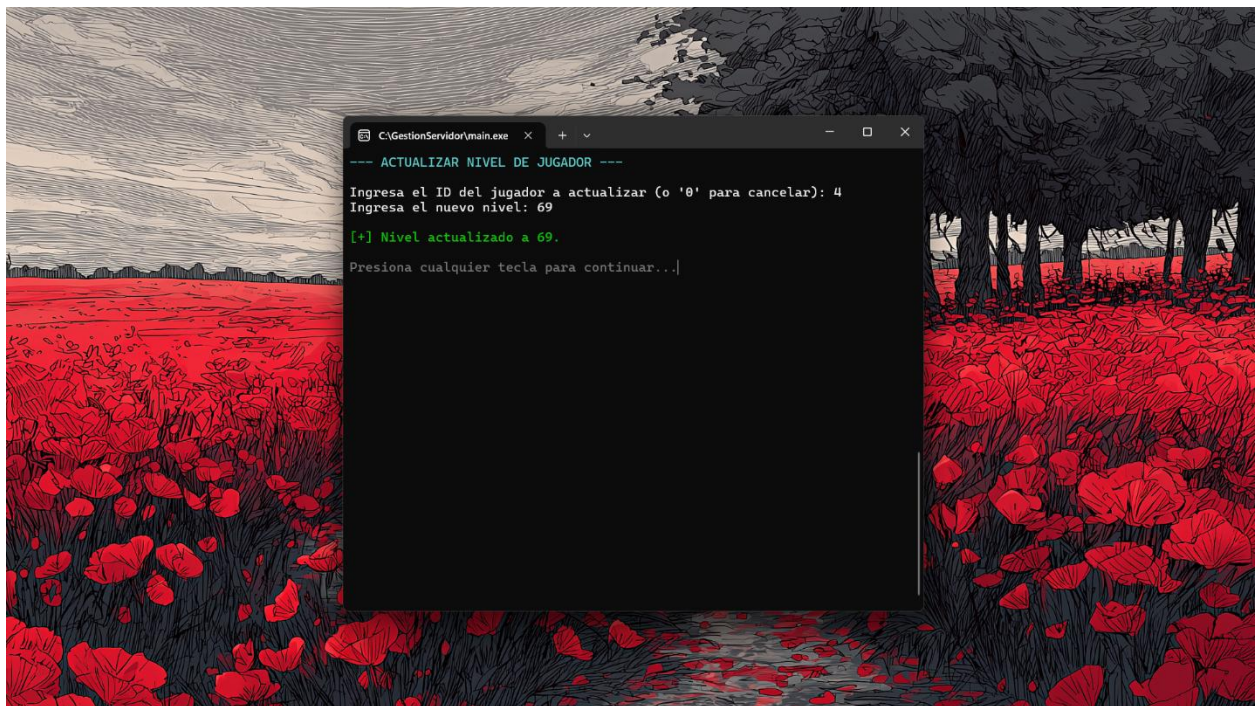
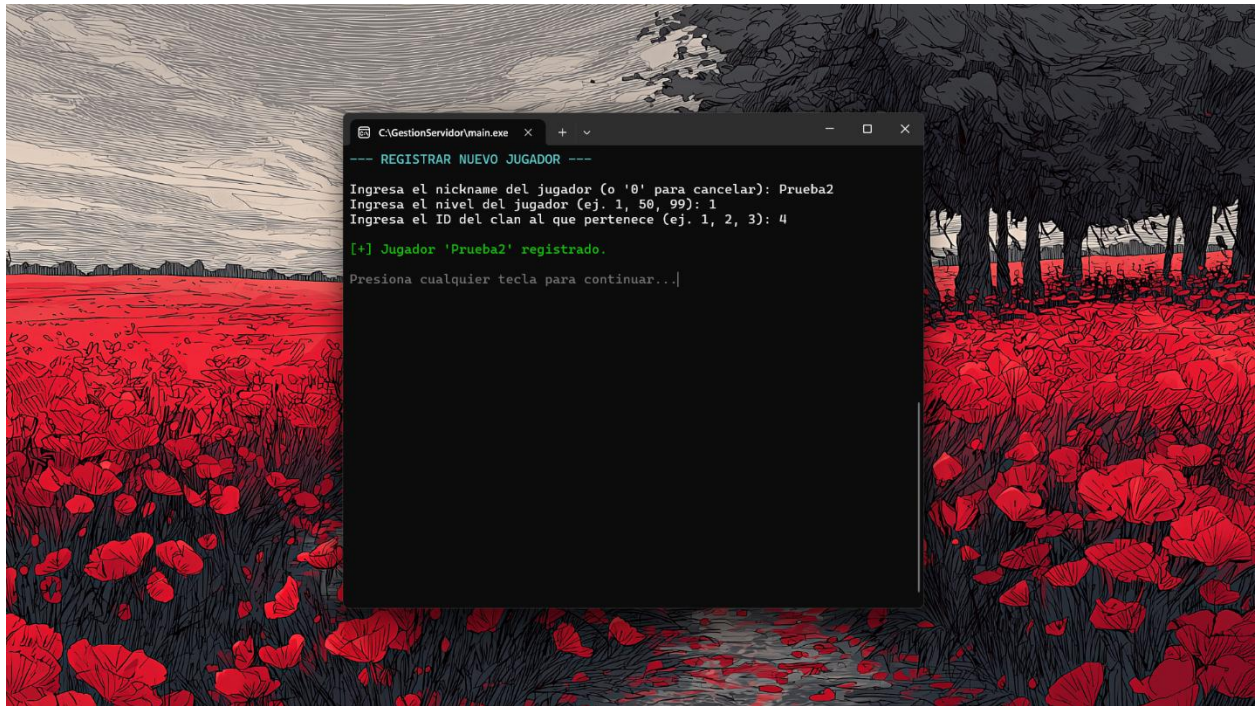
Estructura de Carpetas y Archivos

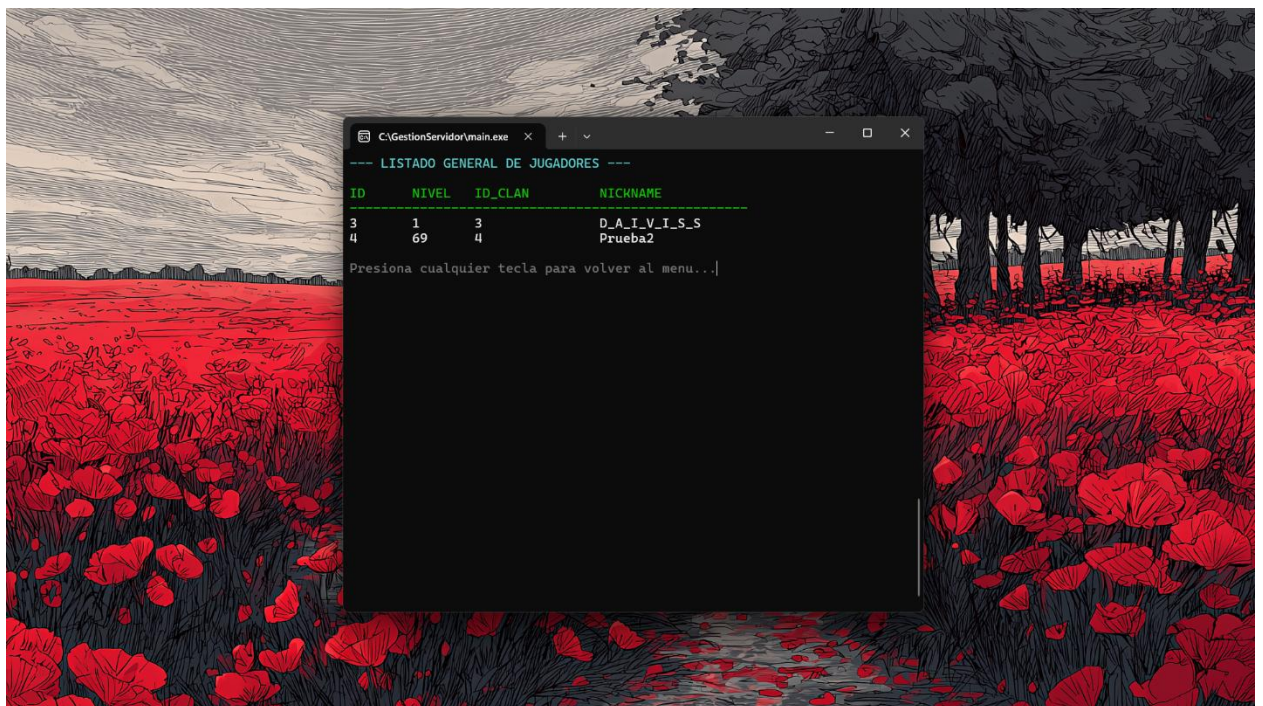
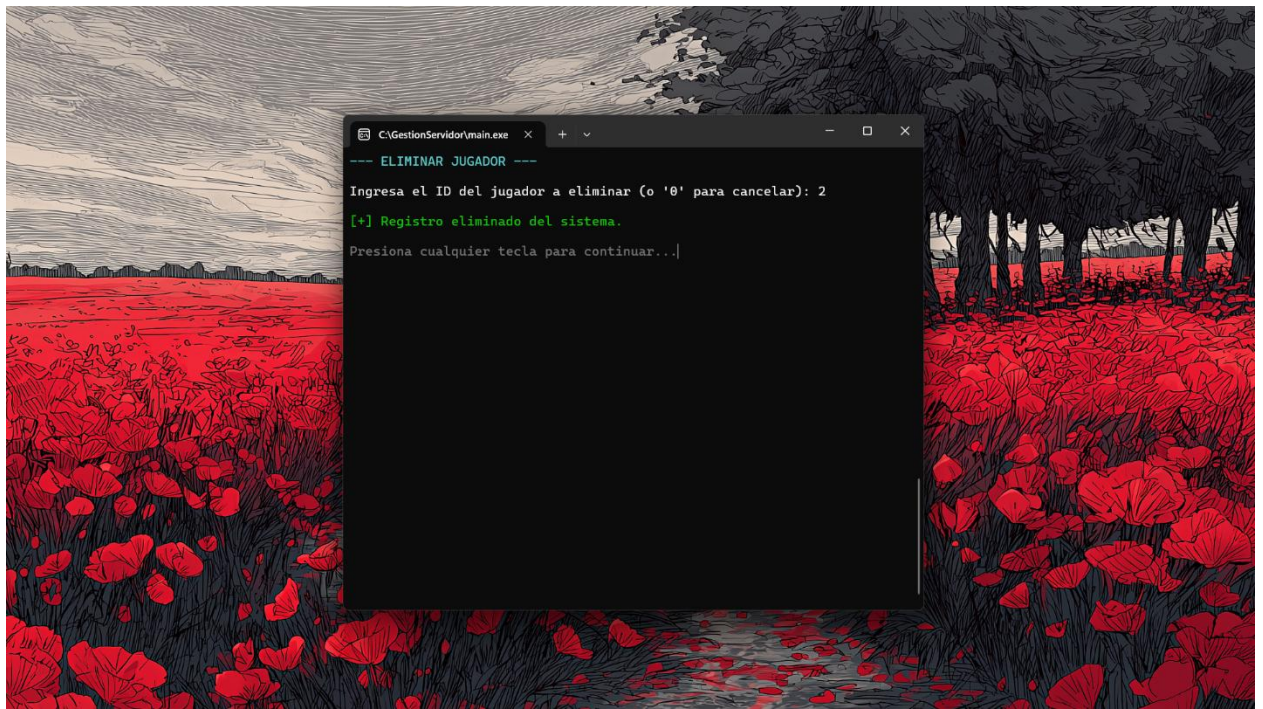
El proyecto se trabajó en Visual Studio Code y está dividido de la siguiente manera:

- ***main.cpp***: Es el archivo principal donde está el código del menú visual y donde se mandan a llamar las funciones para guardar, ver o borrar los datos.
- ***Archivos .h (EloquentORM, MySQLModel, MySQLConexion)***: Son las librerías del framework que usamos para poder conectar C++ con MySQL más fácilmente.
- ***script.sql***: Contiene las instrucciones para crear la base de datos y las tablas que usa el programa.
- ***.gitignore***: Un archivo de configuración para que Git ignore el ejecutable “.exe”, y solo suba el código fuente a la nube.

Evidencias:







Unnamed/control_servidor_db(clanes) - HeidiSQL 12.16.0.7229

Archivo Editar Display Buscar Consulta Herramientas Ira Ayuda

Unnamed Base de datos: control_servidor_db Tabla: clanes Datos Consulta

control_servidor_db.clanes: 4 filas en total (exacto)

#	1 id	2 nombre	3 descripcion
1	1	Grimo Alfa	Clan principal de pruebas
2	2	Beta Force	Segundo clan de soporte
3	3	LosIntaxError	Prueba de primer clan para agregar ad
4	4	PruebaPaEvidencia	Prueba para poder subir captura de evidencia e...

Filtro: Expresión regular

```

24 /* Escalando controles a DPI de pantalla: 125% */
25 SELECT * FROM `information_schema`.`COLUMNS` WHERE TABLE_SCHEMA='control_servidor_db' AND TABLE_NAME='clanes' ORDER BY ORDINAL_POSITION;
26 SHOW INDEXES FROM `clanes` FROM `control_servidor_db`;
27 SELECT * FROM information_schema.REFERENTIAL_CONSTRAINTS WHERE CONSTRAINT_SCHEMA='control_servidor_db' AND TABLE_NAME='clanes' AND REFERENCED_TABLE_NAME IS NOT NULL;
28 SELECT * FROM information_schema.KEY_COLUMN_USAGE WHERE TABLE_SCHEMA='control_servidor_db' AND TABLE_NAME='clanes' AND REFERENCED_TABLE_NAME IS NOT NULL;
29 SHOW COLLATION;
30 SHOW ENGINES;
31 SHOW CREATE TABLE `control_servidor_db`.`clanes`;
32 SELECT tc.CONSTRAINT_NAME, cc.CHECK_CLAUSE FROM `information_schema`.`CHECK_CONSTRAINTS` AS cc, `information_schema`.`TABLE_CONSTRAINTS` AS tc WHERE tc.CONSTRAINT_SCHEMA='control_servidor_db' AND tc.TABLE_NAME='clanes' AND tc.CON
33 SELECT * FROM `control_servidor_db`.`clanes` LIMIT 1000;
  
```

r1 : c2 Conectado: 00:00 h MySQL 8.0.45 Activo durante: 2 días, 00:33 h Hora del servidor: 07:40 Preparado.

Unnamed/control_servidor_db(jugadores) - HeidiSQL 12.16.0.7229

Archivo Editar Display Buscar Consulta Herramientas Ira Ayuda

Unnamed Base de datos: control_servidor_db Tabla: jugadores Datos Consulta

control_servidor_db.jugadores: 2 filas en total (exacto)

#	1 id	2 nickname	3 nivel	4 clan_id
1	3	D_A_I_V_I_S_S	1	3
2	4	Prueba2	69	4

Filtro: Expresión regular

```

31 SHOW CREATE TABLE `control_servidor_db`.`clanes`;
32 SELECT tc.CONSTRAINT_NAME, cc.CHECK_CLAUSE FROM `information_schema`.`CHECK_CONSTRAINTS` AS cc, `information_schema`.`TABLE_CONSTRAINTS` AS tc WHERE tc.CONSTRAINT_SCHEMA='control_servidor_db' AND tc.TABLE_NAME='clanes' AND tc.CON
33 SELECT * FROM `information_schema`.`COLUMNS` WHERE TABLE_SCHEMA='control_servidor_db' AND TABLE_NAME='jugadores' ORDER BY ORDINAL_POSITION;
34 SHOW INDEXES FROM `jugadores` FROM `control_servidor_db`;
35 SELECT * FROM information_schema.REFERENTIAL_CONSTRAINTS WHERE CONSTRAINT_SCHEMA='control_servidor_db' AND TABLE_NAME='jugadores' AND REFERENCED_TABLE_NAME IS NOT NULL;
36 SELECT * FROM information_schema.KEY_COLUMN_USAGE WHERE TABLE_SCHEMA='control_servidor_db' AND TABLE_NAME='jugadores' AND REFERENCED_TABLE_NAME IS NOT NULL;
37 SHOW CREATE TABLE `control_servidor_db`.`jugadores`;
38 SELECT tc.CONSTRAINT_NAME, cc.CHECK_CLAUSE FROM `information_schema`.`CHECK_CONSTRAINTS` AS cc, `information_schema`.`TABLE_CONSTRAINTS` AS tc WHERE tc.CONSTRAINT_SCHEMA='control_servidor_db' AND tc.TABLE_NAME='jugadores' AND tc.C
39 SELECT * FROM `control_servidor_db`.`jugadores` LIMIT 1000;
  
```

r1 : c2 Conectado: 00:00 h MySQL 8.0.45 Activo durante: 2 días, 00:33 h Hora del servidor: 07:41 Preparado.

Código Fuente y Base de Datos:

C++ Code - Script

```
1  -- Universidad Mariano Gálvez
2  -- Facultad de Ingeniería en Sistemas
3  -- Curso: Programación 1
4  -- Autor: David Barrios
5  -- Fecha: 19 de mayo de 2026
6  -- Descripción: Script inicial de base de datos para el control de un servidor de
   MAINKLA (Proyecto Final)
7
8  CREATE DATABASE IF NOT EXISTS control_servidor_db;
9  USE control_servidor_db;
10
11  -- =====
12  -- Entidad Padre: clanes (Grupos organizados en el servidor)
13  -- =====
14  CREATE TABLE clanes (
15      id INT AUTO_INCREMENT PRIMARY KEY,
16      nombre VARCHAR(50) NOT NULL,
17      descripcion VARCHAR(150)
18  );
19
20  -- =====
21  -- Entidad Hija: jugadores (Usuarios registrados)
22  -- Cumple con la regla obligatoria de FK: tabla_id (clan_id)
23  -- =====
24  CREATE TABLE jugadores (
25      id INT AUTO_INCREMENT PRIMARY KEY,
26      nickname VARCHAR(50) NOT NULL,
27      nivel INT DEFAULT 1,
28      clan_id INT,
29      FOREIGN KEY (clan_id) REFERENCES clanes(id)
30  );
31
32  -- Registros iniciales base para pruebas de conexión posteriores
33  INSERT INTO clanes (nombre, descripcion) VALUES ('Gremio Alfa', 'Clan principal de
   pruebas'), ('Beta Force', 'Segundo clan de soporte');
34  INSERT INTO jugadores (nickname, nivel, clan_id) VALUES ('Deivtz', 99, 1),
   ('PlayerOne', 10, 2);
```


C++ Code – main.cpp

```
1  /**
2   * @file main.cpp
3   * @brief Proyecto Final - Sistema de gestion para servidor de videojuegos.
4   * @author David Barrios
5   * @date 27 de mayo de 2026
6   *
7   * @details Este modulo contiene el menu principal interactivo y la integracion
8   * del framework EloquentORM para la persistencia de datos en MySQL.
9   */
10
11 #include <iostream>
12 #include <windows.h>
13 #include <time.h>
14 #include <string>
15 #include "MySQLConexion.h"
16 #include "MySQLModel.h"
17 #include "EloquentORM.h"
18
19 using namespace std;
20
21 /**
22  * @brief Enumeracion de codigos de color ANSI para la consola.
23  */
24 enum Colors {
25     BLACK = 0, BLUE = 1, GREEN = 2, CYAN = 3, RED = 4, MAGENTA = 5,
26     BROWN = 6, LGREY = 7, DGREY = 8, LBLUE = 9, LGREEN = 10,
27     LCYAN = 11, LRED = 12, LMAGENTA = 13, YELLOW = 14, WHITE = 15
28 };
29
30 /**
31  * @brief Modifica los atributos de color del texto y fondo en la consola.
32  * * @param background Color de fondo.
33  * @param text Color del texto.
34  */
35 void aplicar_color(int background, int text) {
36     HANDLE consoleHandle = GetStdHandle(STD_OUTPUT_HANDLE);
37     int newColor = text + (background * 16);
38     SetConsoleTextAttribute(consoleHandle, newColor);
39 }
40
41 /**
42  * @brief Limpia la pantalla utilizando secuencias de escape.
43  */
44 void limpiar_pantalla() {
45     cout << "\033[2J\033[H";
46 }
47
48 /**
49  * @brief Introduce una pausa en la ejecucion del hilo actual.
50  * * @param ms Tiempo de espera en milisegundos.
```

```

51     */
52 void esperar_tiempo(int ms) {
53     for(clock_t t = clock(); (clock() - t) * 1000 / CLOCKS_PER_SEC < ms;);
54 }
55
56 /**
57  * @brief Despliega la interfaz del menu principal del sistema.
58  */
59 void mostrar_menu() {
60     limpiar_pantalla();
61     aplicar_color(BLACK, LCYAN);
62     cout << "===== " << endl;
63     cout << "      PANEL DE CONTROL DEL SERVIDOR      " << endl;
64     cout << "===== " << endl;
65
66     aplicar_color(BLACK, WHITE);
67     cout << "\n1. Registrar Clan (Create)" << endl;
68     cout << "2. Registrar Jugador (Create)" << endl;
69     cout << "3. Actualizar Nivel de Jugador (Update)" << endl;
70     cout << "4. Eliminar Jugador (Delete)" << endl;
71     cout << "5. Ver Lista de Jugadores (Read)" << endl;
72     cout << "6. Salir del Sistema" << endl;
73     cout << "\nSelecciona una opcion: ";
74 }
75
76 /**
77  * @brief Punto de entrada de la aplicacion.
78  * * Gestiona el ciclo de vida de la UI de consola, instanciando la conexion
79  * a la base de datos y controlando las peticiones CRUD.
80  * * @return intCodigo de estado final (0 = Exito).
81  */
82 int main() {
83     int opcionElegida = 0;
84     bool sistemaActivo = true;
85
86     limpiar_pantalla();
87     aplicar_color(BLACK, YELLOW);
88     cout << "Iniciando sistema";
89     for(int i = 0; i < 3; i++) {
90         esperar_tiempo(500);
91         cout << ".";
92     }
93     cout << "\n¡Bienvenido, Administrador!" << endl;
94     esperar_tiempo(1000);
95
96     // Inicializacion de la conexion a la capa de persistencia de datos
97     MySQLConexion conexion("root", "Tm05121204SND", "control_servidor_db");
98
99     if (conexion.open()) {
100         aplicar_color(BLACK, LGREEN);
101         cout << "\n[+] Conexion a la base de datos del server exitosa!\n";
102     } else {
103         aplicar_color(BLACK, LRED);

```

```

104         cout << "\n[-] Fallo al conectar a la base de datos. Revisa tus
credenciales.\n";
105     }
106     esperar_tiempo(3000);
107
108     // Ciclo de control principal
109     while (sistemaActivo) {
110         mostrar_menu();
111         cin >> opcionElegida;
112
113         // Limpieza del stream de entrada para evitar loops por caracteres invalidos
114         if(cin.fail()) {
115             cin.clear();
116             cin.ignore(10000, '\n');
117             opcionElegida = 0;
118         }
119
120         switch (opcionElegida) {
121             case 1: { // Modulo: Create (Clanes)
122                 limpiar_pantalla();
123                 aplicar_color(BLACK, LCYAN);
124                 cout << "--- REGISTRAR NUEVO CLAN ---" << endl;
125                 aplicar_color(BLACK, WHITE);
126
127                 string nombreClan, descClan;
128
129                 cout << "\nIngresa el nombre del clan (o '0' para cancelar): ";
130                 getline(cin >> ws, nombreClan);
131
132                 if (nombreClan == "0") {
133                     aplicar_color(BLACK, YELLOW);
134                     cout << "\n[!] Operacion cancelada. Regresando al menu..." <<
endl;
135                     Sleep(1000);
136                     break;
137                 }
138
139                 if (nombreClan.empty()) {
140                     aplicar_color(BLACK, LRED);
141                     cout << "\n[-] Error: El nombre no puede ser nulo." << endl;
142                     cout << "\nPresiona Enter para volver al menu...";
143                     system("pause>nul");
144                     break;
145                 }
146
147                 cout << "Ingresa una breve descripcion: ";
148                 getline(cin, descClan);
149
150                 EloquentORM clan(conexion, "clanes", {"nombre", "descripcion"});
151                 clan.set("nombre", nombreClan);
152                 clan.set("descripcion", descClan);
153
154                 if(clan.save()) {
155                     aplicar_color(BLACK, LGREEN);

```



```

156         cout << "\n[+] Clan '" << nombreClan << "' guardado exitosamente."
157     << endl;
158     } else {
159         aplicar_color(BLACK, LRED);
160         cout << "\n[-] Excepcion al ejecutar la transaccion." << endl;
161     }
162
163     aplicar_color(BLACK, DGREY);
164     cout << "\nPresiona cualquier tecla para continuar...";
165     system("pause>nul");
166     break;
167 }
168 case 2: { // Modulo: Create (Jugadores)
169     limpiar_pantalla();
170     aplicar_color(BLACK, LCYAN);
171     cout << "--- REGISTRAR NUEVO JUGADOR ---" << endl;
172     aplicar_color(BLACK, WHITE);
173
174     string nicknameJugador, nivelJugador, idClan;
175
176     cout << "\nIngresa el nickname del jugador (o '0' para cancelar): ";
177     getline(cin >> ws, nicknameJugador);
178
179     if (nicknameJugador == "0") {
180         aplicar_color(BLACK, YELLOW);
181         cout << "\n[!] Operacion cancelada." << endl;
182         Sleep(1000);
183         break;
184     }
185
186     if (nicknameJugador.empty()) {
187         aplicar_color(BLACK, LRED);
188         cout << "\n[-] Error: El nickname es un campo obligatorio." <<
189     endl;
190
191         cout << "\nPresiona Enter para volver al menu...";
192         system("pause>nul");
193         break;
194     }
195
196     cout << "Ingresa el nivel del jugador (ej. 1, 50, 99): ";
197     getline(cin, nivelJugador);
198
199     cout << "Ingresa el ID del clan al que pertenece (ej. 1, 2, 3): ";
200     getline(cin, idClan);
201
202     EloquentORM jugador(conexion, "jugadores", {"nickname", "nivel",
203     "clan_id"});
204
205     jugador.set("nickname", nicknameJugador);
206     jugador.set("nivel", nivelJugador);
207     jugador.set("clan_id", idClan);
208
209     if(jugador.save()) {
210         aplicar_color(BLACK, LGREEN);

```

```

206             cout << "\n[+] Jugador '" << nicknameJugador << "' registrado." <<
endl;
207         } else {
208             aplicar_color(BLACK, LRED);
209             cout << "\n[-] Error de integridad referencial. (Verifique
clan_id)." << endl;
210         }
211
212         aplicar_color(BLACK, DGREY);
213         cout << "\nPresiona cualquier tecla para continuar...";
214         system("pause>nul");
215         break;
216     }
217     case 3: { // Modulo: Update (Jugadores)
218         limpiar_pantalla();
219         aplicar_color(BLACK, LCYAN);
220         cout << "--- ACTUALIZAR NIVEL DE JUGADOR ---" << endl;
221         aplicar_color(BLACK, WHITE);
222
223         string entradaId;
224         int idJugador;
225         string nuevoNivel;
226
227         cout << "\nIngresa el ID del jugador a actualizar (o '0' para
cancelar): ";
228         cin >> entradaId;
229
230         if (entradaId == "0") {
231             aplicar_color(BLACK, YELLOW);
232             cout << "\n[!] Operacion cancelada." << endl;
233             Sleep(1000);
234             break;
235         }
236
237         idJugador = stoi(entradaId);
238
239         cout << "Ingresa el nuevo nivel: ";
240         getline(cin >> ws, nuevoNivel);
241
242         EloquentORM jugador(conexion, "jugadores", {"id", "nickname", "nivel",
"clan_id"});
243
244         if(jugador.find(idJugador)) {
245             jugador.set("nivel", nuevoNivel);
246             if(jugador.save()) {
247                 aplicar_color(BLACK, LGREEN);
248                 cout << "\n[+] Nivel actualizado a " << nuevoNivel << "." <<
endl;
249             } else {
250                 aplicar_color(BLACK, LRED);
251                 cout << "\n[-] Error durante la actualizacion." << endl;
252             }
253         } else {
254             aplicar_color(BLACK, LRED);

```

```

255         cout << "\n[-] El registro con ID " << idJugador << " no existe."
<< endl;
256     }
257
258     aplicar_color(BLACK, DGREY);
259     cout << "\nPresiona cualquier tecla para continuar...";
260     system("pause>nul");
261     break;
262 }
263 case 4: { // Modulo: Delete (Jugadores)
264     limpiar_pantalla();
265     aplicar_color(BLACK, LCYAN);
266     cout << "--- ELIMINAR JUGADOR ---" << endl;
267     aplicar_color(BLACK, WHITE);
268
269     string entradaId;
270     int idJugador;
271
272     cout << "\nIngresa el ID del jugador a eliminar (o '0' para cancelar):
";
273     cin >> entradaId;
274
275     if (entradaId == "0") {
276         aplicar_color(BLACK, YELLOW);
277         cout << "\n[!] Operacion cancelada." << endl;
278         Sleep(1000);
279         break;
280     }
281
282     idJugador = stoi(entradaId);
283
284     EloquentORM jugador(conexion, "jugadores", {"id", "nickname", "nivel",
"clan_id"});
285
286     if(jugador.find(idJugador)) {
287         if(jugador.remove()) {
288             aplicar_color(BLACK, LGREEN);
289             cout << "\n[+] Registro eliminado del sistema." << endl;
290         } else {
291             aplicar_color(BLACK, LRED);
292             cout << "\n[-] Fallo en la ejecucion del comando DELETE." <<
endl;
293         }
294     } else {
295         aplicar_color(BLACK, LRED);
296         cout << "\n[-] El registro con ID " << idJugador << " no existe."
<< endl;
297     }
298
299     aplicar_color(BLACK, DGREY);
300     cout << "\nPresiona cualquier tecla para continuar...";
301     system("pause>nul");
302     break;
303 }

```

```

304         case 5: { // Modulo: Read All (Jugadores)
305             limpiar_pantalla();
306             aplicar_color(BLACK, LCYAN);
307             cout << "--- LISTADO GENERAL DE JUGADORES ---" << endl;
308             aplicar_color(BLACK, WHITE);
309
310             EloquentORM jugador(conexion, "jugadores", {"id", "nickname", "nivel",
"clan_id"});
311             vector<map<string, string>> listaJugadores = jugador.getAll();
312
313             if (listaJugadores.empty()) {
314                 aplicar_color(BLACK, YELLOW);
315                 cout << "\n[!] No existen registros en la entidad 'jugadores'." <<
endl;
316             } else {
317                 aplicar_color(BLACK, LGREEN);
318                 cout << "\nID\tNIVEL\tID_CLAN\t\tNICKNAME" << endl;
319                 cout << "-----" <<
endl;
320                 aplicar_color(BLACK, WHITE);
321
322                 // Iteracion sobre el conjunto de resultados retornados por el ORM
323                 for (auto &fila : listaJugadores) {
324                     cout << fila["id"] << "\t"
325                         << fila["nivel"] << "\t"
326                         << fila["clan_id"] << "\t\t"
327                         << fila["nickname"] << endl;
328                 }
329             }
330
331             aplicar_color(BLACK, DGREY);
332             cout << "\nPresiona cualquier tecla para volver al menu...";
333             system("pause>nul");
334             break;
335         }
336         case 6: // Modulo: Salida Segura
337             aplicar_color(BLACK, GREEN);
338             cout << "\nFinalizando sesion... ¡Hasta luego!" << endl;
339             sistemaActivo = false;
340             Sleep(1000);
341             break;
342         default: // Validacion de entrada fuera de rango
343             aplicar_color(BLACK, RED);
344             cout << "\nError: Opcion no contemplada en el menu." << endl;
345             Sleep(1500);
346             break;
347     }

```



```
348     }
349
350     aplicar_color(BLACK, WHITE);
351     return 0;
352 }
```

Repositorio:

<https://github.com/Deivtz/GestionServidor>